How their system updates your database

Your website keeps the registration. Their system reports confirmed payment and attendance automatically.

1 GET THEIR DOCS

Ask for API / webhook docs, sample events and a test account.

2 SHARE ONE REFERENCE

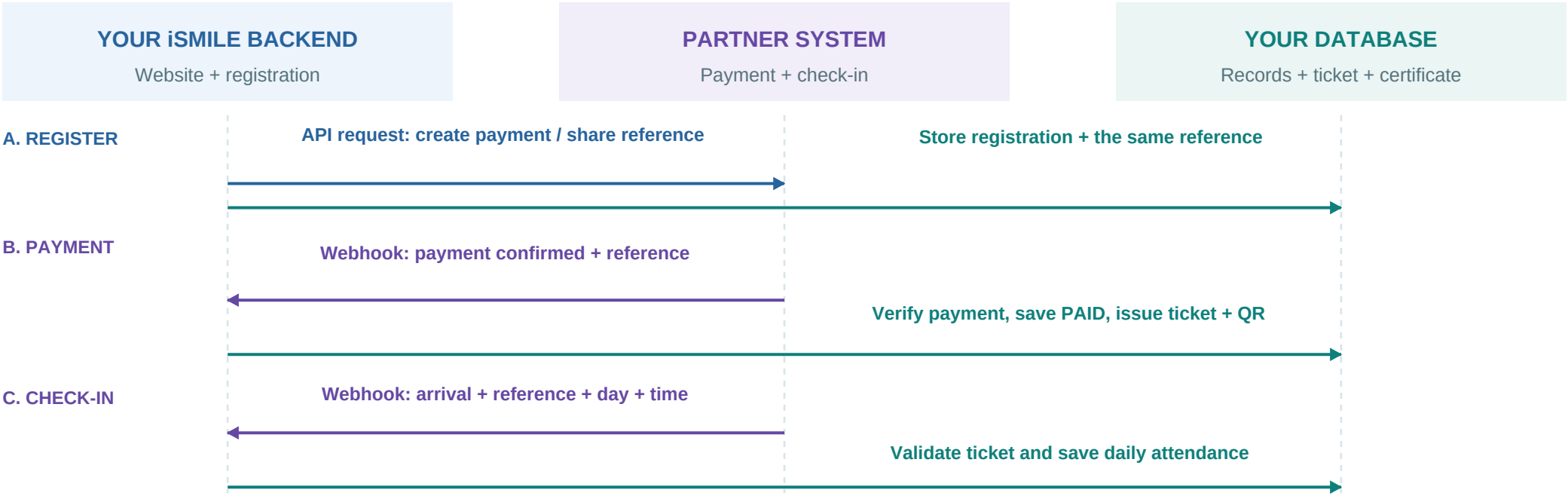
Example: ISM26-000123. Use it in both systems.

3 SAVE THEIR UPDATES

Payment confirmation + check-in for each event day.

4 TEST THE JOURNEY

Register, pay, receive ticket, check in, receive certificate.



CERTIFICATE: valid paid ticket + the agreed attendance rule. Payment alone is not enough.

What we exchange and what we test

All references below are examples. The partner must confirm their real field names, routes and authentication.

API: your backend asks their system

Send: your registration reference, expected amount and currency; ticket ID when their check-in system needs it.
Send name / email only if their documented flow requires it.
They return: their payment ID / payment link or ticket mapping.

WEBHOOK: their system tells your backend

They send a signed notification when something happens.
Your backend checks the sender, reference and event details.
It saves the update, then acknowledges success.
A retry with the same event ID must not create duplicates.

PAYMENT CONFIRMATION - example

event_id: PAY-EVENT-987
reference: ISM26-000123
payment_id: PARTNER-PAY-456
status: confirmed
amount/currency: agreed ticket total

CHECK-IN CONFIRMATION - example

event_id: CHECKIN-EVENT-321
reference: ISM26-000123
ticket_id: linked ticket / QR reference
event_day: 20 November 2026
checked_in_at: original arrival time + timezone

THE TEST: ONE PERSON, ONE REFERENCE, FROM START TO FINISH

Registration > confirmed payment > ticket + QR > Day 1 / Day 2 attendance > eligible certificate

Also test: failed payment, unknown reference, repeated webhook and a delayed check-in.
Agree before launch: does the certificate require attendance on one day or both days?